

Problem Set 3 — Answer Key

Part 1: Predicting Net Financial Assets

1.1 Train/Test Split

A good answer uses `train_test_split` with `random_state=42` and `test_size=0.2`. All preprocessing (e.g., scaling for Ridge/LASSO) must be fit **only on the training set** and then applied to the test set — fitting a scaler on the full data before splitting is a data contamination error and should be penalized.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = data.drop(columns=['net_tfa'])
y = data['net_tfa']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Scale AFTER splitting
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

1.2 Model Comparison Bar Chart

A good answer: - Uses `cross_val_score` or `GridSearchCV` with `cv=5` **on the training set only** to select `alpha` for Ridge and LASSO, and `max_depth` for Random Forest. - Computes RMSE on the held-out test set for all four final models. - Produces a clean bar chart labeled with model names and RMSE values.

Expected pattern of results: OLS will have a reasonable but not great RMSE given the noisy outcome. Ridge and LASSO will perform similarly to OLS or slightly better because the controls are few and relatively low-dimensional — heavy regularization is not very helpful here. Random Forest should achieve the lowest test RMSE because `net_tfa` has a nonlinear, skewed distribution that a linear model cannot fully capture.

A student who finds OLS outperforming Random Forest should double-check that they tuned `max_depth` correctly and did not overfit by using a very deep tree without CV.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.ensemble import RandomForestRegressor
```

```

from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error

models = {
    'OLS': LinearRegression(),
    'Ridge': RidgeCV(alphas=np.logspace(-3, 4, 50), cv=5),
    'LASSO': LassoCV(cv=5, max_iter=10_000),
    'Random Forest': GridSearchCV(
        RandomForestRegressor(n_estimators=200, random_state=42),
        param_grid={'max_depth': [3, 5, 10, None]},
        cv=5, scoring='neg_mean_squared_error'
    ),
}

rmsees = {}
for name, model in models.items():
    X_tr = X_train_sc if name in ('Ridge', 'LASSO') else X_train
    X_te = X_test_sc if name in ('Ridge', 'LASSO') else X_test
    model.fit(X_tr, y_train)
    preds = model.predict(X_te)
    rmsees[name] = np.sqrt(mean_squared_error(y_test, preds))

fig, ax = plt.subplots()
ax.bar(rmsees.keys(), rmsees.values())
ax.set_ylabel('Test RMSE ($)')
ax.set_title('Model Comparison: Predicting Net Financial Assets')
plt.tight_layout()

```

1.3 Learning Curve

A good answer computes training and test RMSE at 10+ points from ~5% to 100% of the training data and plots both curves on the same axes.

Expected shape and interpretation: With a dataset of this size (~9,900 observations), a well-specified Random Forest (or whichever model won) will show:

- **Training RMSE** starting very low at small sample sizes (overfitting to few observations) and rising as more data is added.
- **Test RMSE** starting high and falling as training size grows, then flattening out.

If the two curves converge at a *high* RMSE value, this is a **high-bias** (underfitting) signal — the model's functional form is too simple for the data. If training RMSE stays much lower than test RMSE even at large sample sizes, this is a **high-variance** (overfitting) signal. For a well-tuned Random Forest on this dataset, students should observe that the two curves converge to a reasonably low value, indicating the model is well-fit but that adding more data would still help modestly (the curves may not have fully plateaued).

A student who sees very large gap between train and test RMSE that does not close likely did not constrain `max_depth` enough (or used `max_depth=None`, which will overfit).

```
train_sizes = np.linspace(0.05, 1.0, 12)
train_rmses, test_rmses = [], []

best_model = ... # whichever won step 2

for frac in train_sizes:
    n = int(frac * len(X_train))
    X_sub, y_sub = X_train.iloc[:n], y_train.iloc[:n]
    best_model.fit(X_sub, y_sub)
    train_rmses.append(np.sqrt(mean_squared_error(y_sub,
best_model.predict(X_sub))))
    test_rmses.append(np.sqrt(mean_squared_error(y_test,
best_model.predict(X_test))))

plt.plot(train_sizes * len(X_train), train_rmses, label='Train RMSE')
plt.plot(train_sizes * len(X_train), test_rmses, label='Test RMSE')
plt.xlabel('Training set size')
plt.ylabel('RMSE ($)')
plt.legend()
plt.title('Learning Curve')
```

Part 2: Causal Inference with Double Machine Learning

2.1 Naive OLS

```
import statsmodels.formula.api as smf

controls = ['age', 'inc', 'fsize', 'educ', 'pira', 'married', 'two_earner', 'db']
formula = 'net_tfa ~ e401 + ' + ' + '.join(controls)
ols_result = smf.ols(formula, data=data).fit()
print(ols_result.summary())
```

Expected result: The coefficient on `e401` will be large and statistically significant (roughly \$8,000–\$12,000 depending on specification), but this is almost certainly biased upward. Eligible workers tend to have higher income and better financial habits that also predict higher asset accumulation independently of 401(k) access. A student who simply reports the number without questioning it has missed the point.

2.2 Naive LASSO

```
from sklearn.linear_model import LassoCV
import statsmodels.api as sm

lasso = LassoCV(cv=5, max_iter=10_000)
X_controls = data[controls + ['e401']]
lasso.fit(StandardScaler().fit_transform(X_controls), data['net_tfa'])

# Identify selected variables (non-zero coefficients)
selected = X_controls.columns[lasso.coef_ != 0].tolist()

# Post-LASSO OLS
post_formula = 'net_tfa ~ ' + ' + '.join(selected)
post_lasso_result = smf.ols(post_formula, data=data).fit()
print(post_lasso_result.summary())
```

Expected result: LASSO will likely keep most or all controls given the small number of variables. The coefficient on `e401` will be similar to Naive OLS but potentially even more biased. As discussed in the causal ML lecture, LASSO optimizes for prediction, not causal identification — if a confounder has a weak effect on the outcome but a strong effect on treatment, LASSO may drop it and cause omitted variable bias. The standard error from post-LASSO OLS is also invalid because it ignores the model selection step (no inference correction).

2.3 Double Machine Learning

```
import doubleml as dml
from doubleml import DoubleMLPLR
from sklearn.ensemble import RandomForestRegressor

obj_dml_data = dml.DoubleMLData(
    data,
    y_col='net_tfa',
    d_cols='e401',
    x_cols=controls
)

ml_l = RandomForestRegressor(n_estimators=200, max_depth=5, random_state=42)
ml_m = RandomForestRegressor(n_estimators=200, max_depth=5, random_state=42)

dml_plr = DoubleMLPLR(obj_dml_data, ml_l=ml_l, ml_m=ml_m, n_folds=5)
dml_plr.fit()
print(dml_plr.summary())
```

Expected result: The DML estimate of the ATE should be notably smaller than the Naive OLS estimate (likely in the range of \$6,000–\$9,000) with a tighter, more credible confidence interval. The key insight is that DML orthogonalizes both Y and D with respect to X before running the final regression, removing the confounding influence of controls in a nonparametric way.

2.4 Coefficient Plot

A good coefficient plot shows all three estimates as points (or horizontal bars) with 95% confidence interval whiskers on the same scale, clearly labeled. The visual should make it obvious that the three estimates differ in magnitude, and that the DML confidence interval, while overlapping with OLS, reflects a different — and more credible — identification strategy.

```
estimates = {
    'Naive OLS': (ols_result.params['e401'],
ols_result.conf_int().loc['e401']),
    'Naive LASSO': (post_lasso_result.params['e401'],
post_lasso_result.conf_int().loc['e401']),
    'DML': (dml_plr.coef[0], dml_plr.confint().values[0]),
}

fig, ax = plt.subplots(figsize=(6, 3))
for i, (name, (coef, ci)) in enumerate(estimates.items()):
    ax.errorbar(coef, i, xerr=[[coef - ci[0]], [ci[1] - coef]],
                fmt='o', capsize=5)
    ax.text(coef, i + 0.15, f'${coef:,.0f}', ha='center', fontsize=8)
ax.set_yticks(range(len(estimates)))
ax.set_yticklabels(estimates.keys())
ax.axvline(0, color='grey', linestyle='--', linewidth=0.8)
ax.set_xlabel('Estimated ATE of 401(k) Eligibility on Net Financial Assets ($)')
plt.tight_layout()
```

2.5 Written Interpretation

A full-credit answer should make the following points:

1. **Why estimates differ:** The naive OLS estimate conflates the causal effect of 401(k) eligibility with selection — eligible workers tend to be higher earners with stronger savings propensities. Controlling for observables linearly (OLS) does not fully remove this if the relationship between controls and outcome or controls and treatment is nonlinear.
2. **Role of the nuisance model:** DML uses a flexible ML model (Random Forest) to partial out the effect of controls X from both Y and D separately. The residuals $\tilde{Y} = Y - \hat{E}[Y | X]$ and $\tilde{D} = D - \hat{E}[D | X]$ are then regressed on each other. Because Random Forest captures nonlinearities and interactions, this partialling-out is more complete than what OLS can do with a linear specification.

3. **Key identifying assumption:** The estimate is valid under **conditional unconfoundedness** (selection on observables) — that is, conditional on X , the assignment of 401(k) eligibility is as good as random. If there are unobserved confounders (e.g., unobserved savings preferences that also affect employer 401(k) offerings), even DML cannot recover the true causal effect. This is an assumption about the data-generating process, not something we can verify statistically.

A student who only says “DML is better because it uses machine learning” without explaining the orthogonalization logic or the identifying assumption should receive partial credit.

Common Mistakes to Watch For

Mistake	Where it appears	Why it matters
Fitting scaler/imputer on full data before splitting	Part 1.1	Data contamination; test RMSE is optimistically biased
Tuning hyperparameters on the test set	Part 1.2	Same contamination problem; violates the golden rule
Reporting training RMSE as if it were test RMSE	Part 1.2	Masks overfitting entirely
Including <code>e401</code> in the LASSO before the post-selection step	Part 2.2	LASSO may shrink the treatment variable itself
Not using <code>n_folds ≥ 2</code> in <code>DoubleMLPLR</code>	Part 2.3	Single fold = no cross-fitting; valid inference requires cross-fitting
Claiming DML “proves” causality	Part 2.5	DML is only as valid as the unconfoundedness assumption